

Docker + AWS Deployment — Complete Notes

Project: Real-time collaborative code editor · Deployed with Docker on AWS ECS · Source: Sheryians Coding School

1. What We Build

A **real-time collaborative code editor** (like Google Docs, but for code). Multiple users connect to one server via **Socket.io**, edit code in a **Monaco Editor** (same editor used in VS Code), and see each other's changes instantly.



The Core Problem: Simultaneous Edits

If two users edit the **same line at the same time**, whose change wins? This is solved using a **CRDT** (Conflict-free Replicated Data Type) strategy — used by Notion, Figma, Google Docs.

Yjs — a high-performance CRDT library for building collaborative apps. It automatically merges simultaneous edits without conflicts.

Term	Meaning
Y.Doc	A shared document object — stores all files/text as key-value pairs
Y.Text	Represents a single file's text content inside a Y.Doc
y-socket.io	Connects the Yjs client to the server using Socket.io under the hood
y-monaco (MonacoBinding)	Binds the Monaco Editor UI to the Yjs document, so edits sync automatically
awareness	Tracks which users are currently connected + their state (e.g. username)

When a user types, Yjs computes only the **delta** (the small change) — not the whole document — and broadcasts that delta to the server, which forwards it to all other connected users.

2. Frontend & Backend Setup

Frontend (React + Vite + Tailwind)

- `npm create vite@7` → React app
- Install Tailwind CSS for styling
- Install `@monaco-editor/react` for the code editor UI
- Layout: left sidebar (connected users list) + right panel (Monaco editor)

Backend (Node.js + Express + Socket.io)

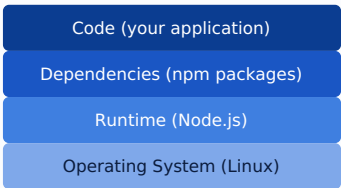
- `npm init -y` → install `express`, `socket.io`, `y-socket.io`
- Create HTTP server, wrap Express app, attach Socket.io
- Add health-check routes (`/` , `/health`) — return status only, no real data, used to verify the server is alive (e.g. AWS dashboards check this)
- Initialize `YSocketIO` on the server → sets up the CRDT sync layer

Users in the Room (Awareness Feature)

1. User enters a username before joining the editor (stored in state + URL query param, so it survives page reload)
2. On connect: `provider.awareness.setLocalStateField('user', username)`
3. Listen for changes: `provider.awareness.on('change', ...)` → read `Array.from(provider.awareness.getStates().values())` to get the list of connected users
4. On page unload: set the user's awareness field to `null` and disconnect the provider (cleanup)

3. Why Docker?

A running server is really made of **4 parts**:



Deploying manually on **AWS EC2** means: create a VM, pick an OS, install Node.js manually, install dependencies manually, copy code manually, then run it — lots of manual work and version-mismatch risk ("works on my machine" problem).

Docker's job: package all 4 layers (OS + runtime + dependencies + code) into a single portable unit called an **image**. Since the OS layer only needs enough to access hardware (no UI needed), a full image is typically just **150-400 MB**.

Once you have this image, **AWS ECS** (Elastic Container Service) can pick it up and run it directly as a working server — no manual setup needed.

4. Docker Basics

Installing Docker

Download **Docker Desktop** (Mac/Windows) from [docker.com](#). Verify install with:

```
docker --version
```

Dockerfile Keywords

Keyword	Purpose
FROM	Base image to start from, e.g. <code>node:20-alpine</code> (Node.js 20 + lightweight Linux, ~5 MB base)
WORKDIR	Sets the working directory inside the image for following commands
COPY	Copies files/folders from your machine into the image
RUN	Executes a command while building the image (e.g. <code>npm install</code>)
CMD	Command that runs when the container starts (e.g. <code>node server.js</code>)

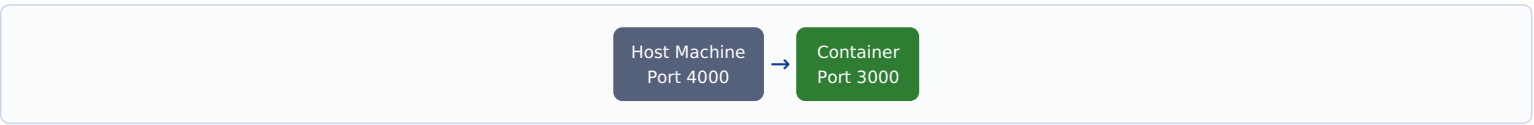
.dockerignore — exclude files that shouldn't be baked into the image, typically `node_modules` (different OS → different binaries) and `.env` (secrets should never live inside an image).

Basic Dockerfile Example (backend only)

```
FROM node:20-alpine
COPY backend/ /app
WORKDIR /app
RUN npm install
CMD ["node", "server.js"]
```

Build & Run Commands

Command	What it does
<code>docker build -t backend .</code>	Builds an image from the Dockerfile, tags it "backend"
<code>docker run backend</code>	Runs the image as a container (a mini isolated environment)
<code>docker run -p 4000:3000 backend</code>	Port mapping: forwards host port 4000 → container's internal port 3000



Why port mapping is needed: a container is an isolated mini-environment inside your OS. By default you can't reach what's running inside it — the `-p` flag routes traffic from a host port to the container's internal port.

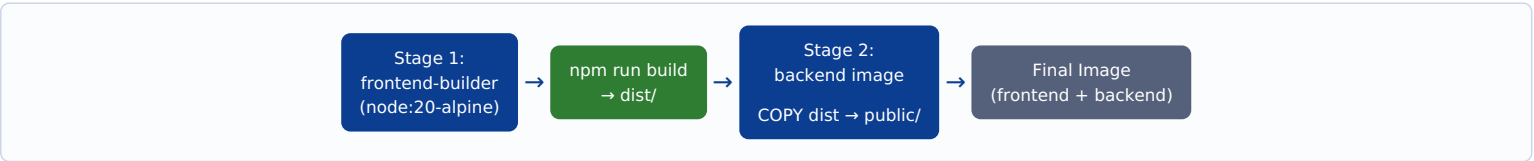
5. Serving Frontend + Backend on One Domain

In production, frontend and backend should be served from the **same domain** (not separate dev ports like 5173 and 3000).

- Run `npm run build` in the frontend → creates a `dist/` folder with just 3 files: `index.html` , a CSS file, a JS file (the entire React app bundled)
- Copy the contents of `dist/` into the backend's `public/` folder
- In `server.js` , add middleware: `app.use(express.static('public'))`
- Now visiting the backend's URL directly serves the frontend too

6. Multi-Stage Docker Build

Since we need to (a) build the frontend, and (b) copy that build into the backend, we use a **multi-stage build** — the standard production pattern.



```
# Stage 1 – build frontend
FROM node:20-alpine AS frontend-builder
COPY frontend/ /app
WORKDIR /app
RUN npm install
RUN npm run build          # produces /app/dist

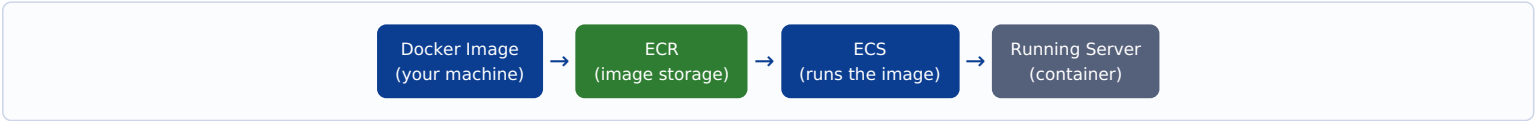
# Stage 2 – build backend, embed frontend build
FROM node:20-alpine
```

```
COPY backend/ /app
WORKDIR /app
RUN npm install
COPY --from=frontend-builder /app/dist ./public
CMD ["node", "server.js"]
```

Build & test locally:

```
docker build -t server . then docker run -p 4000:3000 server
```

7. AWS: ECR & ECS



Service	Role
ECR (Elastic Container Registry)	Cloud storage for your Docker images
ECS (Elastic Container Service)	Pulls the image from ECR and runs it as a live server

Step 1 — Create a Restricted IAM User

Never use your **root/admin** AWS credentials for pushing images — if leaked, your entire AWS infrastructure is at risk. Instead, create a dedicated IAM user with only the permissions it needs.

- AWS Console → IAM → Users → Create User (e.g. `docker-aws-video`)
- Attach policies: **AmazonEC2ContainerRegistryFullAccess** (ECR) + **AmazonECS_FullAccess** (ECS)
- Generate an **Access Key** for this user (for CLI/programmatic access)

Step 2 — Install & Configure AWS CLI Locally

- Mac: `brew install awscli` | Windows: use the official MSI installer
- Verify: `aws --version`
- Run `aws configure` → enter Access Key ID, Secret Access Key, Region (e.g. `ap-northeast-1` for Tokyo), output format `json`

This links your local machine's CLI to the restricted IAM user — every AWS request from your terminal now acts on that user's behalf.

Step 3 — Handle CPU Architecture Mismatch

Important gotcha: Images built on Apple Silicon (Mac M-series) default to `arm64` architecture, but AWS ECS machines typically run on `x86_64` (`amd64`) — the more optimized/standard architecture. An arm64 image will **fail to run** on an x86_64 ECS task.

Fix: build a cross-platform image using **Docker Buildx**:

```
docker buildx build --platform linux/amd64 -t docker-aws-server . --load
```

Step 4 — Push Image to ECR

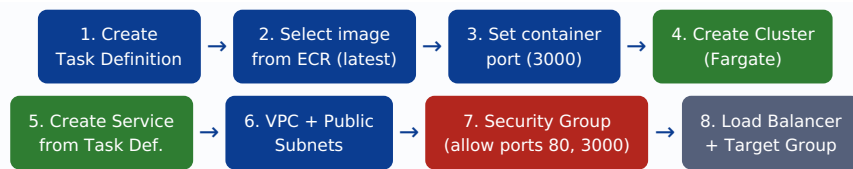
- ECR Console → Create Repository (e.g. `docker-aws`)
- Click "View push commands" — copy the 4 commands shown:
 - `aws ecr get-login-password ...` — authenticate Docker to ECR
 - `docker build -t <repo> .` — build (or use buildx as above)
 - `docker tag <repo> <ecr-url>/<repo>` — rename for ECR
 - `docker push <ecr-url>/<repo>` — upload the image

8. Deploying on AWS ECS

Key Concepts

Term	Meaning
Task Definition	Blueprint: which image to run + how much CPU/RAM/ports it needs
Service	Actually runs (executes) the Task Definition, keeps it alive
Cluster	A logical group where your services run (e.g. Fargate — serverless, no manual server management)
VPC (Virtual Private Cloud)	Your own private network space on AWS
Subnet	A sub-network inside a VPC — public subnets are internet-accessible, private subnets are not
Security Group	Firewall rules — controls which ports/traffic are allowed in and out
ALB (Application Load Balancer)	Routes internet traffic to the right container/port; gives you a public DNS to access your app
Target Group	Tells the ALB which port on the container to forward traffic to

Deployment Flow



Step-by-Step

1. **Task Definition:** ECS Console → Task Definitions → Create → choose **AWS Fargate** (serverless — no manual server/instance management) → set CPU/memory → add container: name it, pick the ECR image (`:latest`), map container port **3000**
2. **Task Role vs Task Execution Role:** Execution Role = permissions needed *while starting up* (e.g. pull image from ECR). Task Role = permissions the *running* server itself has to access other AWS services. Both can reuse the same IAM role with ECR access.
3. **Cluster:** Create a cluster (Fargate only) to hold your services
4. **VPC:** Create a new VPC (choose "VPC and more" so subnets + Internet Gateway are auto-configured) — needed because the restricted IAM user usually lacks permission to create one, so use your root/admin account for this one-time setup
5. **Security Group:** Allow inbound traffic on **port 80** (HTTP) and **port 3000** (app) from `0.0.0.0/0` (anywhere) — a very common cause of "can't access my deployed app" is a missing/incorrect security group rule
5. **Load Balancer:** Create an Application Load Balancer → internet-facing → attach to your VPC's public subnets → create a Target Group pointing at container port 3000
7. **Create Service:** Attach the cluster, task definition, VPC, **public** subnets (not private — private subnets can't be reached from the internet), security group, and load balancer
3. Once deployed, copy the **ALB's DNS name** from the service's networking tab — this is the public URL to access your app (later map a custom domain to it via your DNS provider)

Common troubleshooting checklist if the app doesn't load:

- Is the container in a **public** subnet? (private subnets are unreachable from the internet)
- Does the **Security Group** allow inbound traffic on the app's port from anywhere?
- Was the image built for the correct **CPU architecture** (amd64, not arm64)?

9. Summary — What Docker Actually Solves

Docker's real value: it locks the **exact OS version + exact runtime version** into the image. So whatever environment you develop in is *exactly* the environment that runs in production — eliminating the classic "*works on my machine, not on yours*" problem.

Your Code + Dependencies + Node.js Runtime + Linux OS = Portable Docker Image

Notes compiled from: "Complete Docker + AWS | Deploy Your First Project in One Shot (Beginners)" — Sheryians Coding School